

Actividad de laboratorio: introducción práctica a sistemas P2P

Laboratorio TIC — DIINF

Curso: Sistemas Distribuidos y Paralelos
Departamento de Ingeniería Informática — USACH

10 de julio de 2026

1. Objetivo de aprendizaje

En una hora, cada grupo montará una mini-red P2P entre sus nodos, repartirá un archivo en piezas, implementará una estrategia de descarga y medirá tiempos y mensajes ante fallas. El foco está en **experimentar y medir**, no solo en revisar teoría.

2. Rol del docente

El docente orienta con **teoría** y responde dudas si las solicitan. **No** prepara archivos ni configura la red: eso lo hace cada grupo.

3. Repositorio y fork

Código base y este enunciado están en el repositorio público del curso:

https://github.com/miguelcarcamov/lab_experiences

Cada grupo debe:

1. Hacer **fork** del repositorio a su cuenta de GitHub.
2. Clonar **su fork** en al menos una máquina del grupo:

```
git clone git@github.com:<usuario>/lab_experiences.git
cd lab_experiences/p2p-intro
```

3. Implementar `select_next_piece` en `node.py` y commitear en **master** de su fork.
4. No subir artefactos generados en el lab (`demo.bin`, `payload/`, `piezas/`); el `.gitignore` ya los excluye.

4. Organización

- **Lugar:** Laboratorio TIC, DIINF.
- **Duración:** 60 minutos.
- **Grupos:** 3 equipos de 5 estudiantes (los mismos del laboratorio del curso).
- **Modalidad:** cada grupo usa varios nodos del lab (recomendado: al menos 3 por grupo). Deben conectarlos en la misma red local (switch) y anotar la IP de cada máquina.

5. Materiales y prerequisites

- Nodos del laboratorio TIC y switch para interconectarlos.
- Python 3 (solo librería estándar).
- Carpeta `p2p-intro/` del fork (archivos principales):
 - `node.py` — servidor, protocolo y `fetch` listos; **TODO:** `select_next_piece(...)`.
 - `discover.py` — descubrimiento TCP, listo para usar.
 - `prepare_payload.py` — generan y trocean el archivo de prueba del grupo.
 - `activity_1.pdf` — enunciado (en el repositorio; la fuente LaTeX la mantiene el curso).
- Papel para dibujar el grafo de conectividad.

5.1. Preparación del archivo (lo hace el grupo)

1. Generar un archivo de prueba: `python3 prepare_payload.py --gen-demo` (crea `demo.bin`, ~48 KB).
2. Trocearlo y repartir piezas entre los nodos del grupo, por ejemplo con 4 nodos:

```
python3 prepare_payload.py --input demo.bin --chunk-size 4096 --nodes 4
```

Esto produce `manifest.json` y carpetas `payload/nodes/node01/`, ... con subconjuntos de `chunk.NNN.bin`.

3. Copiar en cada PC su carpeta como `./piezas/` (cada nodo arranca con **parte** del archivo, nadie con el archivo completo).
4. Crear `seeds.txt` y `peers.txt` (formato `host:puerto:node-id` por línea; ver abajo).

5.2. seeds.txt y peers.txt: en qué se diferencian

Ambos archivos listan nodos del grupo, pero **no cumplen el mismo rol**.

- **seeds.txt (semillas)** — puntos de entrada para **descubrimiento**.
 - Lo usa `discover.py` en la fase inicial (0–10 min).
 - Basta con **2–3 nodos conocidos** del grupo; no hace falta listar a todos.
 - Para cada semilla, el script envía PING por TCP y reporta si responde, su `node_id` y qué piezas tiene localmente.
 - Sirve para **verificar conectividad** y dibujar el grafo observado.
 - Analogía: nodos *bootstrap* de BitTorrent o “vecinos que ya conozco” para el primer contacto.
- **peers.txt (peers)** — fuentes de descarga para **reconstruir el archivo**.
 - Lo usa `node.py fetch` durante la difusión (10–40 min).
 - Debe incluir **todos los nodos del grupo** que puedan servir piezas (típicamente todos los que corren `serve`).
 - En cada iteración, `fetch` consulta a *cada* peer de esta lista (`LIST_PIECES`) y luego `select_next_piece` elige de cuál pedir la siguiente pieza.
 - Si un nodo no está en `peers.txt`, **nunca** se le pedirán fragmentos, aunque responda en `discover.py`.

Formato común (una entrada por línea; líneas con `#` se ignoran):

```
192.168.1.10:8800:pc01
192.168.1.11:8800:pc02
```

Ejemplo para un grupo de 5 nodos: `seeds.txt` puede tener solo `pc01` y `pc02`; `peers.txt` debe listar `pc01` a `pc05`.

Comandos:

```
python3 discover.py --node-id pc01 --seeds-file seeds.txt
python3 node.py fetch --manifest manifest.json --peers peers.txt \
  --pieces ./piezas --out recuperado.bin
```

La reconstrucción es correcta si `recuperado.bin` tiene el mismo SHA-256 que `manifest.json`.

6. Qué hace el software actual y qué falta

6.1. Qué ya está implementado (no tocar)

El archivo `node.py` entrega la infraestructura completa de un peer:

- **Servidor TCP concurrente** (`node.py serve`): escucha en un puerto, acepta conexiones y atiende cada una en un hilo aparte. Publica las piezas que el nodo tiene localmente en `./piezas/`.

- **Protocolo de mensajes** en JSON (una línea por mensaje):
 - PING → PONG con `node_id` y lista de piezas.
 - LIST_PIECES → HAVE_PIECES con los índices disponibles.
 - REQUEST_PIECE{index} → PIECE (datos en base64 + SHA-256) o ERROR.
- **Cliente de descarga** (`node.py fetch`): consulta a los peers de `peers.txt` qué piezas tienen, pide las que faltan, **verifica el hash** de cada pieza y, al completar todas, ensambla el archivo y comprueba el SHA-256 contra `manifest.json`.
- **Manejo básico de fallas**: si un peer no responde (timeout o error de conexión), se marca como caído y el ciclo de descarga continúa con el resto.
- `discover.py`: sondea por TCP las entradas de `seeds.txt` (semillas) y reporta qué nodos responden; no descarga piezas.

6.2. Qué falta (lo implementan ustedes)

El cliente `fetch` sabe *cómo* pedir y verificar una pieza, pero no *cuál* pedir ni *a quién*. Esa decisión es la función:

```
select_next_piece(peers_info, my_pieces, total_pieces)
```

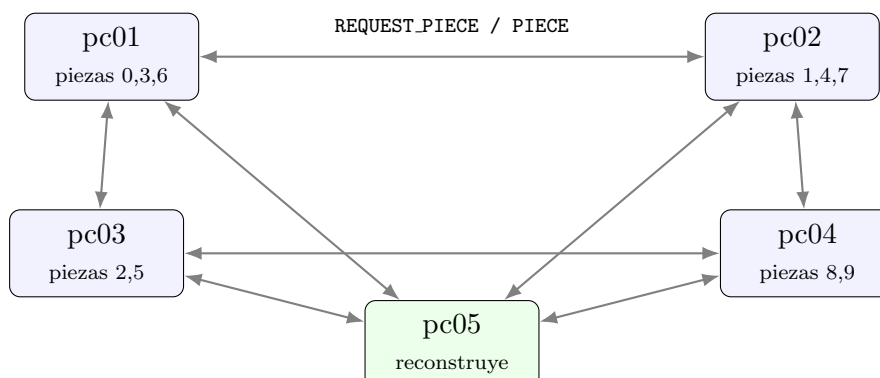
Recibe la lista de peers vivos (con las piezas que tiene cada uno), el conjunto de piezas que este nodo ya posee y el total de piezas. Debe devolver el par (`peer`, `indice`) que se pedirá a continuación, o `None` cuando el archivo esté completo. Hoy lanza `NotImplementedError`: mientras no la completen, `fetch` no funciona.

6.3. Por qué justo esa función

La configuración de red ya está resuelta. Lo pedagógico es **decidir a qué peer pedir qué pieza** (secuencial, *rarest-first*, aleatorio, etc.): esa elección cambia el número de mensajes, el tiempo total y la tolerancia a fallas, que es exactamente lo que van a medir.

6.4. Diagrama de la red de nodos

Cada nodo corre a la vez como **servidor** (publica sus piezas) y como **cliente** (pide las que le faltan). Al inicio ningún nodo tiene el archivo completo: las piezas están repartidas.



Ejemplo ilustrativo: pc05 necesita todas las piezas y las pide a distintos peers según la estrategia de `select_next_piece`. Si un nodo cae (p. ej. pc02), sus piezas deben conseguirse desde otro que también las tenga. El grafo real dependerá de las IP y de cómo repartan las piezas.

7. Desarrollo de la actividad

Tiempo	Fase	Qué hace cada grupo
0–10 min	Fork + red + descubrimiento	Fork y clone del repo; conectar nodos; <code>prepare_payload.py</code> ; copiar piezas/; <code>node.py</code> <code>serve</code> ; <code>seeds.txt/peers.txt</code> ; <code>discover.py</code> y grafo.
10–40 min	Implementación + difusión	10–15 min: completar <code>select_next_piece</code> y commit en su fork. 15–40 min: <code>node.py</code> <code>fetch</code> ...; anotar tiempos y mensajes.
40–50 min	Inyección de falla	Otro grupo detiene 1–2 nodos <code>serve</code> sin aviso. Reintentar <code>fetch</code> ; anotar detección y recuperación.
50–60 min	Cierre con datos	Plenario: estrategia, tiempos, mensajes, grafo, efecto de la falla. URL del fork si el docente lo pide.

7.1. Indicaciones operativas

- Repartan roles (red, payload, implementación, Git, medición, reporte).
- Solo modifiquen `select_next_piece` en `node.py` salvo acuerdo documentado del grupo.
- Si `discover.py` no ve peers, revisen IP, firewall y que `serve` esté activo.
- Documenten la estrategia usada en la medición final.

8. Preguntas de reflexión (cierre grupal)

1. ¿Qué estrategia implementaron en `select_next_piece`? ¿Por qué?
2. ¿Cómo detectaron un nodo caído?
3. ¿Cuántos mensajes y cuánto tiempo antes y después de la falla?
4. Si cayeran la mitad de los nodos, ¿su estrategia seguiría funcionando? ¿Qué faltaría (DHT, réplicas, *rarest-first* global)?

9. Extensión opcional

Quienes terminen antes pueden probar *rarest-first* o esbozar cómo localizarían piezas sin `peers.txt` central.